Рубежный контроль №1 по курсу ПиК ЯП

Вариант Г. Предметная область: Хранимая процедура – База данных (№34)

Текст программы

```
from operator import itemgetter

class DB:
    """База данных"""
    def __init__(self, id, name):
        self.id = id
        self.name = name


class Proc:
    """Хранимая процедура"""
    def __init__(self, id, name, lines_count, db_id):
        self.id = id
        self.name = name
        self.lines_count = lines_count  # количественный признак
        self.db_id = db_id


class ProcDB:
    """
    Связь многие-ко-многим 'Хранимые процедуры - Базы данных'
    """
    def __init__(self, db_id, proc_id):
        self.db_id = db_id
        self.proc_id = proc_id


# Базы данных
dbs = [
    DB(1, 'АудитБД'),
    DB(2, 'АрхивнаяБД'),
    DB(3, 'БухгалтерияБД'),
    DB(4, 'СкладБД'),
]

# Хранимые процедуры
procs = [
    Proc(1, 'sp_CalcBalance', 150, 1),
    Proc(2, 'sp_ArchiveMove', 90, 2),
    Proc(3, 'sp_PostInvoice', 120, 3),
    Proc(4, 'sp_RecalcTax', 80, 3),
    Proc(5, 'sp_WriteOff', 60, 4),
]

# Связь многие-ко-многим
procs_dbs = [
    ProcDB(1, 1),
    ProcDB(2, 2),
    ProcDB(3, 3),
    ProcDB(3, 4),
    ProcDB(4, 5),
    ProcDB(1, 2),  # дополнительно для наглядности связи many-to-many
]


def main():
    """Основная функция"""

    # Соединение данных один-ко-многим (База данных - Хранимая процедура)
    one_to_many = [(p.name, p.lines_count, d.name)
```

```
            for d in dbs
            for p in procs
            if p.db_id == d.id]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [(d.name, pd.db_id, pd.proc_id)
                         for d in dbs
                         for pd in procs_dbs
                         if d.id == pd.db_id]

    many_to_many = [(p.name, p.lines_count, db_name)
                    for db_name, db_id, proc_id in many_to_many_temp
                    for p in procs if p.id == proc_id]

    print('Задание Г1 (один-ко-многим)')
    print('Список баз данных, название которых начинается с буквы "А", и хранимые процедуры в них:')
    res_g1 = {}
    for d in dbs:
        if d.name.startswith('A'):
            d_procs = list(filter(lambda i: i[2] == d.name, one_to_many))
            d_procs_names = [x for x, _, _ in d_procs]
            if d_procs_names:
                res_g1[d.name] = d_procs_names

    for db_name, proc_list in res_g1.items():
        print(f'  {db_name}: {proc_list}')

    print('\nЗадание Г2 (один-ко-многим)')
    print('Список баз данных с максимальным количеством строк хранимых процедур в каждой БД, отсортированный
по максимальному количеству строк:')
    res_g2_unsorted = []
    for d in dbs:
        d_procs = list(filter(lambda i: i[2] == d.name, one_to_many))
        if len(d_procs) > 0:
            d_lines = [lines for _, lines, _ in d_procs]
            max_lines = max(d_lines)
            res_g2_unsorted.append((d.name, max_lines))

    res_g2 = sorted(res_g2_unsorted, key=itemgetter(1))
    print(res_g2)

    print('\nЗадание Г3 (многие-ко-многим)')
    print('Список всех связанных хранимых процедур и баз данных, отсортированный по базам данных:')
    res_g3 = sorted(many_to_many, key=itemgetter(2))
    print(res_g3)


if __name__ == '__main__':
    main()
```

Результаты выполнения программы

Задание Г1 (один-ко-многим)
Список баз данных, название которых начинается с буквы "А", и хранимые процедуры в них:
АудитБД: ['sp_CalcBalance']
АрхивнаяБД: ['sp_ArchiveMove']

Задание Г2 (один-ко-многим)
Список баз данных с максимальным количеством строк хранимых процедур в каждой БД, отсортированный по максимальному количеству строк:
[('СкладБД', 60), ('АрхивнаяБД', 90), ('БухгалтерияБД', 120), ('АудитБД', 150)]

Задание Г3 (многие-ко-многим)
Список всех связанных хранимых процедур и баз данных, отсортированный по базам данных:
[('sp_CalcBalance', 150, 'АудитБД'), ('sp_ArchiveMove', 90, 'АудитБД'), ('sp_ArchiveMove', 90, 'АрхивнаяБД'), ('sp_PostInvoice', 120, 'БухгалтерияБД'), ('sp_RecalcTax', 80, 'БухгалтерияБД'), ('sp_WriteOff', 60, 'СкладБД')]